Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 29.11.2023.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
   a. Vodite računa da imate sve spremno što vam je neophodno.
   b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 30 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2023_Test1_GrupaE_IT1g2021**
8. **Minimalni broj bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka je 5**.
9. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Brisanje elemenata iz steka (Stack Pop)*

Napisati metodu koja implementira standardan način za brisanje elemenata iz steka.

Dodatne napomene za rešavanje zadatka:

- Algoritam napisati u okviru zakomentarisane metode ***pop?(int value).***
- Pomenuta metoda nalazi se u okviru klase ***Stack*** koja se nalazi u pridruženom template-u.
- Obezbediti kod u Main metodi za testiranje algoritma.

Zadatak 2 – *Eliminisanje negativnih brojeva iz niza*

Kreirati funkciju koja će iz statičkog niza eliminisati sve negativne cele brojeve. Algoritam je neophodno napisati sa linearnom vremenskom kompleksnošću O(n).

Dodatni zahtevi za rešavanje zadatka:

- Algoritam mora rezultovati kreiranjem novog statičkog niza.
- Zahtevan potpis metode: ***public static long[] EliminateNegatives (long[] arr)***

Primer:

- Input: $[2, -3, 5, -2, 10]$
- Output: $[2, 5, 10]$
- Input: []
- Output: []

Zadatak 3 – *Krišnamurtijev broj*

Kreirati algoritam koji proverava da li se od elemenata cirkularne liste može kreirati Krišnamurtijev broj. Krišnamurtijev broj je broj čija suma faktorijela svake cifre je jednaka samom broju. Algoritam je neophodno napisati sa linearnom vremenskom kompleksnošću O(n).

Dodatni zahtevi za rešavanje zadatka:

- Neophodno je implementirati funkciju ***Factorial*** koja treba da pronađe faktorijel prosleđenog broja.
- Broj 145 je Kiršnamurtijev broj: 1! + 4! + 5! = 1 + 24 + 120 = 145
- Broj 15 nije Kiršnamurtijev broj: 1! + 5! = 1 + 120 = 121
- Zadatak rešavati pomoću strukture cirkularne liste (klasa *CircularList*) koja je data u okviru šablona zadatka.
- Zahtevan potpis metode: ***public bool IsListKrishnamurthyNumber()***

Primer:

- Input 1: 1 → 4 → 5
- Output 1: true

- Input 2: 1 → 2 → 5
- Output 2: false